

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Lab Assignment 1 (5%) - Indexes

Name	:	Lau Yee Wen
Matric Id	:	A23CS0099

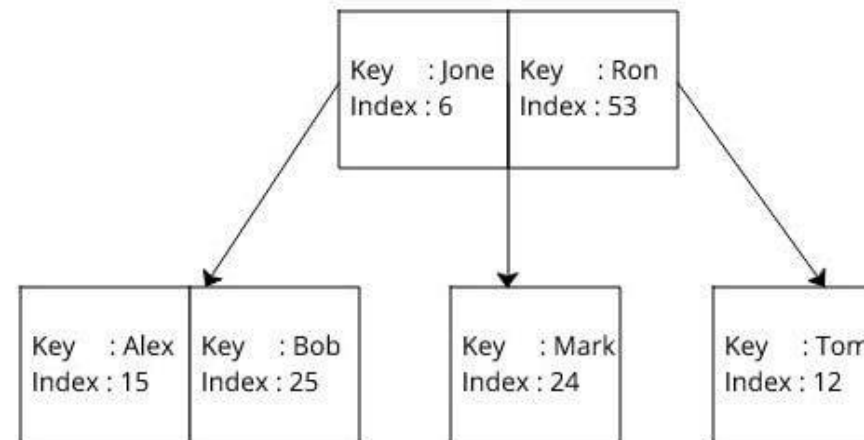
Name	:	Lee Yin Shen
Matric Id	:	A23CS0236

Name	:	Poh Lok Yee
Matric Id	:	A23CS0262

TOPIC:

B-TREE INDEXES

A data structure that organizes data hierarchically (like a tree) to enable fast searching, sorting, and range queries.



WHAT IS B-Tree Index?

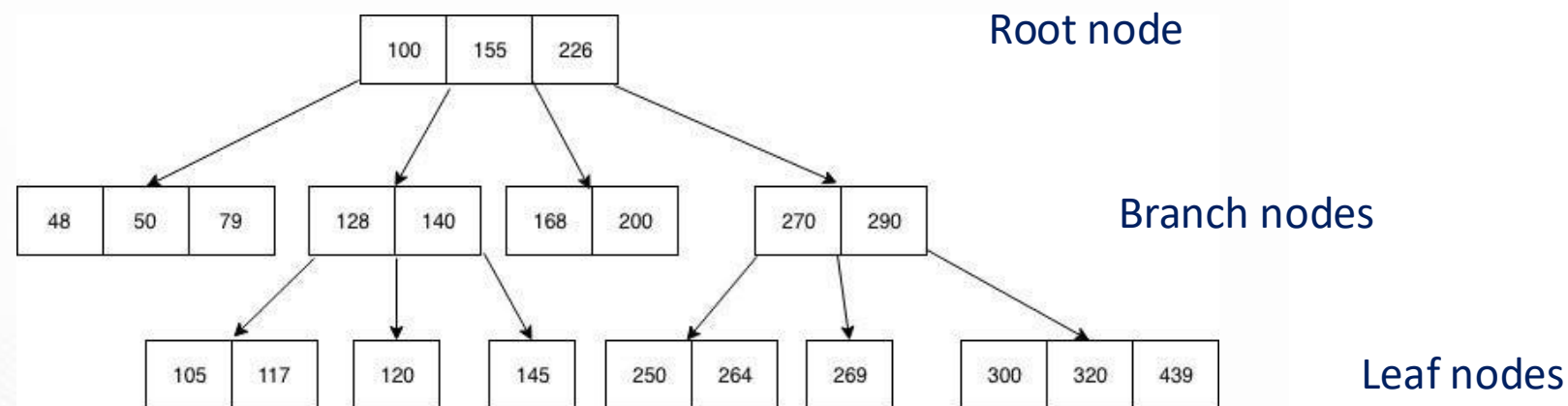
B-Tree (Balanced Tree) index is the default index type used by most database storage engines, including MySQL, Oracle and SQL server.

It organizes data in a hierarchical, tree-like structure with multiple levels of branch nodes that allows for fast data retrieval, insertions, and deletions while keeping the tree balanced.



How it organizes data internally:

- Data is stored in **nodes**, which contain key values and pointers to child nodes.
- The tree is **balanced**, meaning all leaf nodes (bottom level) are at the same depth.
- Nodes contain **sorted key values**, allowing efficient binary-like search.
- Search starts at the **root node**, moves through intermediate branches, and reaches the **leaf nodes** containing pointers to actual table rows.
- Since data is sorted, B-Tree indexes support **fast range scanning**, such as BETWEEN, <, >, and sorting operations.



Advantages	Disadvantages
• Efficient Range Queries: Excellent for BETWEEN, >, < operators.	• Storage Overhead: Indexes require extra disk space.
• Fast Lookups & Sorting: Ideal for exact matches (=) and speeds up ORDER BY.	• Slower Writes: INSERT, UPDATE, DELETE operations are slower as the index must also be updated.
• Consistent Performance: Remains fast and balanced even as data grows or changes.	

SQL Section

1. Create database

```
1 • CREATE DATABASE index_lab;  
2 • USE index_lab;
```

#	Time	Action	Duration / Fetch	Message
✓ 1	23:51:57	CREATE DATABASE index_lab	0.016 sec	1 row(s) affected
✓ 2	23:51:57	USE index_lab	0.000 sec	0 row(s) affected

Create database 'index_lab', then use 'index_lab', so that all the following SQL commands will be run inside this specific database.

2. Create a table 'employee'

```
1 • CREATE TABLE employee(  
2     emp_id INT,  
3     emp_name VARCHAR(50),  
4     department VARCHAR(30),  
5     salary DECIMAL(10,2),  
6     join_date DATE  
7 );
```

#	Time	Action	Duration / Fetch	Message
✓ 1	00:12:51	CREATE TABLE emplo...	0.016 sec	0 row(s) affected

	emp_id	emp_name	department	salary	join_date
--	--------	----------	------------	--------	-----------

This result shows that the 'employee' table has been successfully created with the specified columns. The database is now ready to store data according to this structure.

3. Insert employee data (16 rows)

```

1 • INSERT INTO employee(emp_id, emp_name, department, salary, join_date) VALUES
2 (1, 'Ali Hassan', 'Sales', 5000.00, '2022-01-15'),
3 (2, 'Siti Aminah', 'IT', 6500.00, '2021-03-20'),
4 (3, 'David Chen', 'Finance', 7200.00, '2020-05-10'),
5 (4, 'Mei Ling', 'Sales', 5200.00, '2022-02-01'),
6 (5, 'John Doe', 'IT', 6800.00, '2021-07-11'),
7 (6, 'Priya Singh', 'Marketing', 4800.00, '2023-01-30'),
8 (7, 'Robert Tan', 'Finance', 7000.00, '2020-11-05'),
9 (8, 'Fatimah Yusof', 'HR', 5500.00, '2021-09-01'),
10 (9, 'Kenji Tanaka', 'IT', 9000.00, '2019-04-12'),
11 (10, 'Lisa Wong', 'Sales', 5100.00, '2022-08-22'),
12 (11, 'Michael B.', 'Marketing', 4900.00, '2023-03-15'),
13 (12, 'Zara Ibrahim', 'HR', 5600.00, '2021-10-10'),
14 (13, 'Daniel Lee', 'Finance', 7300.00, '2021-01-19'),
15 (14, 'Ahmad F.', 'Sales', 2900.00, '2023-05-05'),
16 (15, 'Chloe Lim', 'IT', 6600.00, '2022-06-18'),
17 (16, 'Ravi Kumar', 'Finance', 3500.00, '2023-04-11');
  
```

	emp_id	emp_name	department	salary	join_date
▶	1	Ali Hassan	Sales	5000.00	2022-01-15
	2	Siti Aminah	IT	6500.00	2021-03-20
	3	David Chen	Finance	7200.00	2020-05-10
	4	Mei Ling	Sales	5200.00	2022-02-01
	5	John Doe	IT	6800.00	2021-07-11
	6	Priya Singh	Marketing	4800.00	2023-01-30
	7	Robert Tan	Finance	7000.00	2020-11-05
	8	Fatimah Yusof	HR	5500.00	2021-09-01
	9	Kenji Tanaka	IT	9000.00	2019-04-12
	10	Lisa Wong	Sales	5100.00	2022-08-22
	11	Michael B.	Marketing	4900.00	2023-03-15
	12	Zara Ibrahim	HR	5600.00	2021-10-10
	13	Daniel Lee	Finance	7300.00	2021-01-19
	14	Ahmad F.	Sales	2900.00	2023-05-05
	15	Chloe Lim	IT	6600.00	2022-06-18
	16	Ravi Kumar	Finance	3500.00	2023-04-11

This result confirms that all 16 rows of sample employee data were successfully inserted into the 'employee' table. The table now contains data that we can use to test our index.

#	Time	Action	Duration / Fetch	Message
✓ 1	00:29:02	INSERT INTO employ...	0.015 sec	16 row(s) affected Records: 16 Duplicates: 0 Warnings: 0

4. Add a B-Tree index on 'salary'

1. **CREATE INDEX** idx_salary **ON** employee(salary);

Index: **idx_salary**

Definition:

Type	BTREE
Unique	No
Visible	Yes
Columns	salary

	emp_id	emp_name	department	salary	join_date
▶	9	Kenji Tanaka	IT	9000.00	2019-04-12
	13	Daniel Lee	Finance	7300.00	2021-01-19
	3	David Chen	Finance	7200.00	2020-05-10
	7	Robert Tan	Finance	7000.00	2020-11-05
	5	John Doe	IT	6800.00	2021-07-11
	15	Chloe Lim	IT	6600.00	2022-06-18
	2	Siti Aminah	IT	6500.00	2021-03-20
	12	Zara Ibrahim	HR	5600.00	2021-10-10
	8	Fatimah Yusof	HR	5500.00	2021-09-01
	4	Mei Ling	Sales	5200.00	2022-02-01
	10	Lisa Wong	Sales	5100.00	2022-08-22
	1	Ali Hassan	Sales	5000.00	2022-01-15
	11	Michael B.	Marketing	4900.00	2023-03-15
	6	Priya Singh	Marketing	4800.00	2023-01-30
	16	Ravi Kumar	Finance	3500.00	2023-04-11
	14	Ahmad F.	Sales	2900.00	2023-05-05

#	Time	Action	Duration / Fetch	Message
✓ 1	00:36:16	CREATE INDEX idx_sa...	0.046 sec	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

This result shows the **idx_salary** index was created successfully. This means the database now has a sorted list of all the 'salary' values within the index, allowing it to find any salary or range of salary almost instantly.

5. Execute and record query time

```
1 SELECT * FROM employee WHERE salary BETWEEN 3000 AND 7000;
```

	emp_id	emp_name	department	salary	join_date
▶	1	Ali Hassan	Sales	5000.00	2022-01-15
	2	Siti Aminah	IT	6500.00	2021-03-20
	4	Mei Ling	Sales	5200.00	2022-02-01
	5	John Doe	IT	6800.00	2021-07-11
	6	Priya Singh	Marketing	4800.00	2023-01-30
	7	Robert Tan	Finance	7000.00	2020-11-05
	8	Fatimah Yusof	HR	5500.00	2021-09-01
	10	Lisa Wong	Sales	5100.00	2022-08-22
	11	Michael B.	Marketing	4900.00	2023-03-15
	12	Zara Ibrahim	HR	5600.00	2021-10-10
	15	Chloe Lim	IT	6600.00	2022-06-18
	16	Ravi Kumar	Finance	3500.00	2023-04-11

This result shows the 12 employees whose salaries are within the specified range. The query is very fast(0.000 sec) because the database uses the **idx_salary** B-Tree. It doesn't scan the whole table, but it uses the sorted index to find the start of the range 3000 and read sequentially until >7000.

#	Time	Action	Duration / Fetch	Message
✓ 1	00:48:31	SELECT * FROM empl...	0.000 sec / 0.000 sec	12 rows

Duration / Fetch
0.000 sec / 0.000 sec

6. Add an index on 'join_date' and compare range query performance

```
CREATE INDEX idx_join_date ON employee(join_date);
SELECT * FROM employee WHERE join_date BETWEEN '2021-01-01' AND '2022-01-01';
```

	emp_id	emp_name	department	salary	join_date
▶	13	Daniel Lee	Finance	7300.00	2021-01-19
	2	Siti Aminah	IT	6500.00	2021-03-20
	5	John Doe	IT	6800.00	2021-07-11
	8	Fatimah Yusof	HR	5500.00	2021-09-01
	12	Zara Ibrahim	HR	5600.00	2021-10-10

This result shows the 7 employees who joined in 2020 or 2021. The query is also very fast(0.000 sec) because the database used the new **idx_join_date** index to efficiently find all employees within that date range.

#	Time	Action	Duration / Fetch	Message
✓ 1	01:31:56	SELECT * FROM em...	0.000 sec / 0.000 sec	5 row(s) returned

Duration / Fetch
 0.000 sec / 0.000 sec

7. Use EXPLAIN to confirm index usage.

```
EXPLAIN SELECT * FROM employee WHERE salary BETWEEN 3000 AND 7000;
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	employee	NULL	ALL	idx_salary	NULL	NULL	NULL	16	75.00	Using where

```
EXPLAIN SELECT * FROM employee WHERE join_date BETWEEN '2020-01-01' AND '2021-12-31';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	employee	NULL	range	idx_join_date	idx_join_date	4	NULL	7	100.00	Using index condition

The EXPLAIN output compares the two queries and proves the B-Tree index is working.

For the **join_date query**, the key column clearly shows idx_join_date was used, and the type is range, which is a very fast and efficient way to find the data.

For the **salary query**, the key column is NULL and the type is ALL. This means no index was used. **The query optimizer will always find the most efficient way to run the SQL query.**

8. Drop the index and re-run queries. Record time difference.

```
DROP INDEX idx_join_date ON employee;
```

#	Time	Action	Duration / Fetch	Message
1	02:04:47	SELECT * FROM emp...	0.000 sec / 0.000 sec	rows returned

Duration / Fetch
0.000 sec / 0.000 sec

Dataset too small, can't clearly see the time difference.

```
EXPLAIN SELECT * FROM employee WHERE join_date BETWEEN '2020-01-01' AND '2021-12-31';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	employee	NULL	ALL	NULL	NULL	NULL	NULL	16	11.11	Using where

type	possible_keys	key
ALL	NULL	NULL

Now, the type is ALL and the key is NULL. This means the database is not use the index and forced to perform a full table scan, which is the slowest and least efficient method.

Why is the B-Tree index ideal for range and sorting queries ?

Data organise in:

01 SORTED ORDER Helps Sorting Queries

- data already **stored in sequence**
- database can **read** the **data** directly in **sorted order**

02 BALANCED TREE STRUCTURE Helps Range Queries

- allows database to **quickly locate** the starting point and **sequentially scan** through the next values
- tree is balanced, finding that first point is always fast

THANK YOU



univteknologimalaysia



utm.my



utmofficial